

UNIVERSIDADE FEDERAL DE SERGIPE (UFS)



Bruno Henrique Carneiro da Silva - 202400017620
Diego Carvalho Cavalcante - 202300027196
Fábio Henrique Lisboa de Souza - 202400017746
Flávio Henrique de Jesus Cruz - 202400017782
Guilherme Sollano Andrade dos Santos - 202100045896
José Weverton de Oliveira Vilar - 202600072420
Leonardo Caricchio do Nascimento - 202400018046
Matheus Henrique Silva de Melo - 202300027534

AUDITORIA DE MATURIDADE EM ECOSSISTEMAS LLM

Análise do Projeto Open Source: Vanna AI

Atividade Avaliativa 1 (A1) apresentada à disciplina de Engenharia de Software (COMP0503) da Universidade Federal de Sergipe (UFS), como requisito parcial para obtenção de nota.

Prof. Glauco Carneiro

Repositório GitHub: `ES-2026-2.Vanna`

Aracaju - SE

2026

Conteúdo

1	Introdução e Contextualização	2
2	Equipe de Auditoria e Divisão de Papéis	2
3	Eixo I: Ciclo de Vida e Metodologias Ágeis (Foco: GPR)	4
3.1	Planejamento e Ritmo de Entregas	4
3.2	Gestão de Riscos (Volatilidade de APIs)	4
4	Eixo II: Engenharia de Requisitos (Foco: GRE)	4
4.1	Requisito 1: Correção da persistência de memória no ChromaDB	5
4.2	Requisito 2: Suporte à integração com Gemini LLM	5
4.3	Requisito 3: Correção da integração com PGVector	5
5	Eixo III: Arquitetura e Modelagem (Foco: PJR)	6
5.1	Visão Geral e Desacoplamento da IA	6
5.2	Modelagem UML da Fronteira Arquitetural	7
5.3	Alinhamento com Modelos de Maturidade	7
6	Eixo IV: Verificação e Validação - V&V (Foco: V&V)	8
6.1	Análise de Verificação (Workflows e Testes)	8
6.2	Análise de Validação e Prevenção de Alucinações	9
7	Eixo V: Qualidade de Software (Foco: GQA)	9
7.1	Auditoria do CONTRIBUTING.md	9
7.2	Dívida Técnica Analisada	9
7.3	Avaliação segundo Modelos de Maturidade	10
8	Plano de Melhoria de Processo	10
8.1	Síntese dos Gaps e Critérios de Priorização	11
8.2	Ação 1: Gerência de Requisitos com Rastreabilidade Formal (Foco: GRE)	11
8.3	Ação 2: Gerência de Projetos com Controle de Riscos (Foco: GPR)	11
8.4	Plano de Execução e Métricas	12
8.5	Conclusão do Plano	12

1 Introdução e Contextualização

A presente auditoria técnica analisa o projeto open source **Vanna** (<https://github.com/vanna-ai/vanna>), um ecossistema baseado em Modelos de Linguagem (LLMs) voltado para a geração de consultas SQL dinâmicas. O objetivo deste relatório é avaliar a maturidade dos processos de software adotados no repositório, contrapondo as práticas empíricas da comunidade ágil aos rigorosos padrões de Qualidade estabelecidos pelos modelos **CMMI-DEV (v2.0)** e **MPS.BR (MR-MPS-SW)**.

A auditoria explora cinco eixos fundamentais: Ciclo de Vida e Metodologias Ágeis, Engenharia de Requisitos, Arquitetura e Modelagem, Verificação e Validação (V&V) e Qualidade de Software. Por fim, culmina na elaboração de um Plano de Melhoria de Processo, delineando os passos necessários caso o projeto buscasse uma certificação oficial (MPS.BR Nível G).

2 Equipe de Auditoria e Divisão de Papéis

A equipe atuou de forma coordenada, simulando um ambiente real de auditoria de software, com as seguintes atribuições:

- **Diego Carvalho Cavalcante** - Eixo I (Ágil): Análise de Milestones, roadmap, releases e gestão de riscos de APIs.
- **Bruno Henrique Carneiro da Silva** - Eixo II (Requisitos): Investigação de RFCs e rastreabilidade transversal de requisitos (Issue → PR → Código).
- **Fábio Henrique Lisboa de Souza** - Eixo III (Arquitetura): Avaliação do desacoplamento da camada de IA e elaboração do Diagrama UML estrutural.
- **Leonardo Caricchio do Nascimento** - Eixo IV (V&V): Análise dos workflows de Integração Contínua (CI), estratégias de mitigação de alucinações e processos de Peer Review.
- **José Weverton de Oliveira Vilar** - Eixo V (Qualidade): Auditoria do `CONTRIBUTING.md`, mapeamento da dívida técnica e ferramentas de análise estática.
- **Flávio Henrique de Jesus Cruz** - Plano de Melhoria: Definição estratégica de ações corretivas visando a certificação MPS.BR Nível G.
- **Matheus Henrique Silva de Melo** - Tech Lead: Estruturação do repositório e consolidação técnica dos artefatos, criação, edição e formatação do pdf final em LaTeX.

- **Guilherme Sollano Andrade dos Santos** - Direção de Vídeo: Coordenação da defesa técnica, gravação e condução da narrativa investigativa.

3 Eixo I: Ciclo de Vida e Metodologias Ágeis (Foco: GPR)

O alinhamento deste eixo foca no CMMI (Planejamento, Monitoramento e Controle) e no MPS.BR (Gerência de Projetos - GPR).

3.1 Planejamento e Ritmo de Entregas

O projeto Vanna adota uma abordagem de ciclo de vida altamente adaptativa. O ritmo de entregas é notavelmente agressivo, caracterizado por **78 releases publicadas** registradas no GitHub. Esta alta frequência demonstra uma esteira de Continuous Deployment ágil.

Contudo, sob a ótica da maturidade de processos, constata-se uma carência de planejamento formal de longo prazo:

- **Milestones e Roadmap:** Não há utilização sistemática da feature de "Milestones" no GitHub para agrupar entregas táticas, tampouco um roadmap público que antecipe as visões futuras do produto para a comunidade.
- O rastreamento de tarefas opera quase exclusivamente num fluxo reativo de Issues levantadas por usuários e resolvidas ativamente pelos 72 contribuidores do projeto.

3.2 Gestão de Riscos (Volatilidade de APIs)

Aplicações baseadas em LLM sofrem com riscos técnicos críticos decorrentes da dependência de APIs externas (como OpenAI, Anthropic, Gemini). No repositório do Vanna, a mitigação técnica desse risco ocorre pela adoção de classes abstratas (adapters) que isolam a dependência. Entretanto, **não há um registro formal de riscos** gerenciado pela equipe (Risk Register), caracterizando uma falha em Gerência de Projetos (GPR).

4 Eixo II: Engenharia de Requisitos (Foco: GRE)

Este eixo analisa a capacidade do projeto em manter o alinhamento de requisitos (MPS.BR: Gerência de Requisitos - GRE e CMMI: REQM). A auditoria focou na rastreabilidade (Issue → Pull Request → Código) e na gestão de mudanças. Identificou-se a **ausência de processos formais de RFC (Request for Comments)**. As decisões são tomadas diretamente nas Issues, sinalizando baixa maturidade formal.

4.1 Requisito 1: Correção da persistência de memória no ChromaDB

Rastreabilidade: Issue #1031 → PR #1081

Descrição: Usuários relataram que o sistema não conseguia acessar dados previamente armazenados no ChromaDB, indicando falha na persistência da memória entre execuções.

Análise do Código e Implementação: A solução (PR #1081) envolveu alterações na função de recuperação de coleções (`_get_collection`), englobando:

- Implementação de inicialização tardia (*lazy initialization*) da função de embeddings.
- Correção no processo de recuperação de coleções existentes.
- Substituição do tratamento genérico de exceções por `NotFoundError`.
- Adição de testes automatizados para validar a persistência.

A alteração remove a inicialização da função de embedding ao recuperar coleções existentes, evitando reprocessamento desnecessário. A issue foi rapidamente associada a um PR, demonstrando **boa rastreabilidade técnica**. Contudo, observou-se baixa interação técnica e ausência de discussões aprofundadas prévias, indicando fragilidade no refinamento de requisitos.

4.2 Requisito 2: Suporte à integração com Gemini LLM

Rastreabilidade: Issue #986 → Commit direto (Sem PR associado)

Descrição e Análise: A issue apresenta um processo colaborativo entre usuário e desenvolvedor. Relatou-se um problema de importação do módulo `GeminiLlmService`. O desenvolvedor respondeu rapidamente e forneceu a correção via um commit direto. O usuário validou a correção iterativamente. O fluxo evidencia eficácia ágil, porém a ausência de um PR para controle de revisão, documentação ou RFC indica uma abordagem informal e de baixo controle de mudanças.

4.3 Requisito 3: Correção da integração com PGVector

Rastreabilidade: Issue #659 → PR #660

Descrição: Relato de falhas na integração com `pgvector`, incluindo erros de atributos inexistentes e problemas relacionados a execução assíncrona.

Análise Crítica: Durante a discussão na Issue #659, levantou-se a hipótese de falhas na programação assíncrona. O próprio contribuidor reconheceu a ausência de testes prévios antes do lançamento da feature. Ao analisar o PR #660, o código apresentava **inconsistências arquiteturais**: métodos com a mesma finalidade foram definidos de

forma síncrona e assíncrona gerando conflitos (corrotinas). Houve indícios de sobrescrita na definição de embeddings em tempo de execução. Este requisito exemplifica rastreabilidade completa de um incidente, mas expõe graves falhas no processo de Garantia da Qualidade, justificando a subsequente dívida técnica do projeto.

5 Eixo III: Arquitetura e Modelagem (Foco: PJR)

O Eixo III foca na área de processo **Projeto e Construção (PJR)** do MPS.BR e na área de **Solução Técnica (TS)** do CMMI-DEV v2.0. O objetivo central é avaliar como o *design* técnico do projeto isola a lógica de negócio da volatilidade inerente às APIs externas de Modelos de Linguagem de Larga Escala (LLMs).

5.1 Visão Geral e Desacoplamento da IA

O Vanna AI adota uma arquitetura inspirada no padrão **Microkernel** (ou Arquitetura de Plug-ins). O *core* do sistema (o agente orquestrador) atua como o *Kernel*, definindo os contratos e fluxos de execução, enquanto as integrações com diferentes provedores de IA (OpenAI, Anthropic, Gemini, etc.) funcionam como plug-ins intercambiáveis.

A investigação no código-fonte apontou o uso intencional de mecanismos rigorosos de design para garantir este desacoplamento:

- **Abstract Base Classes (ABC) e *Strategy Pattern*:** O projeto utiliza o padrão *Strategy* para permitir a troca dinâmica de provedores. No núcleo do sistema (`src/vanna/core/llm/base.py`), a classe abstrata `LlmService` define métodos contratuais obrigatórios, como `send_request` e `stream_request`. Isso assegura que o sistema dependa de abstrações e não de implementações concretas, respeitando o Princípio da Inversão de Dependência (SOLID). As integrações específicas (ex: `OpenAILlmService` e `AnthropicLlmService`) realizam este contrato.
- **Injeção de Dependência (DI):** A classe orquestradora principal, `Agent` (`src/vanna/core/agent/agent.py`), não instancia um provedor diretamente. A inicialização exige que um objeto do tipo `LlmService` seja injetado via parâmetro (`__init__(self, llm_service: LlmService)`), delegando o acoplamento externo para a camada de configuração.
- **Uso de DTOs (*Data Transfer Objects*):** Para blindar a camada de negócio contra os esquemas proprietários de cada API (como o formato JSON de mensagens da OpenAI), o sistema trafega dados usando DTOs internos estritos (`LlmRequest` e `LlmResponse`). As classes de *Strategy* atuam como tradutoras finais.

5.2 Modelagem UML da Fronteira Arquitetural

Para evidenciar o isolamento da lógica de negócio, foi modelado o Diagrama de Classes UML (Figura 1), com foco estrito na fronteira estrutural entre o **Agent** e os SDKs externos.

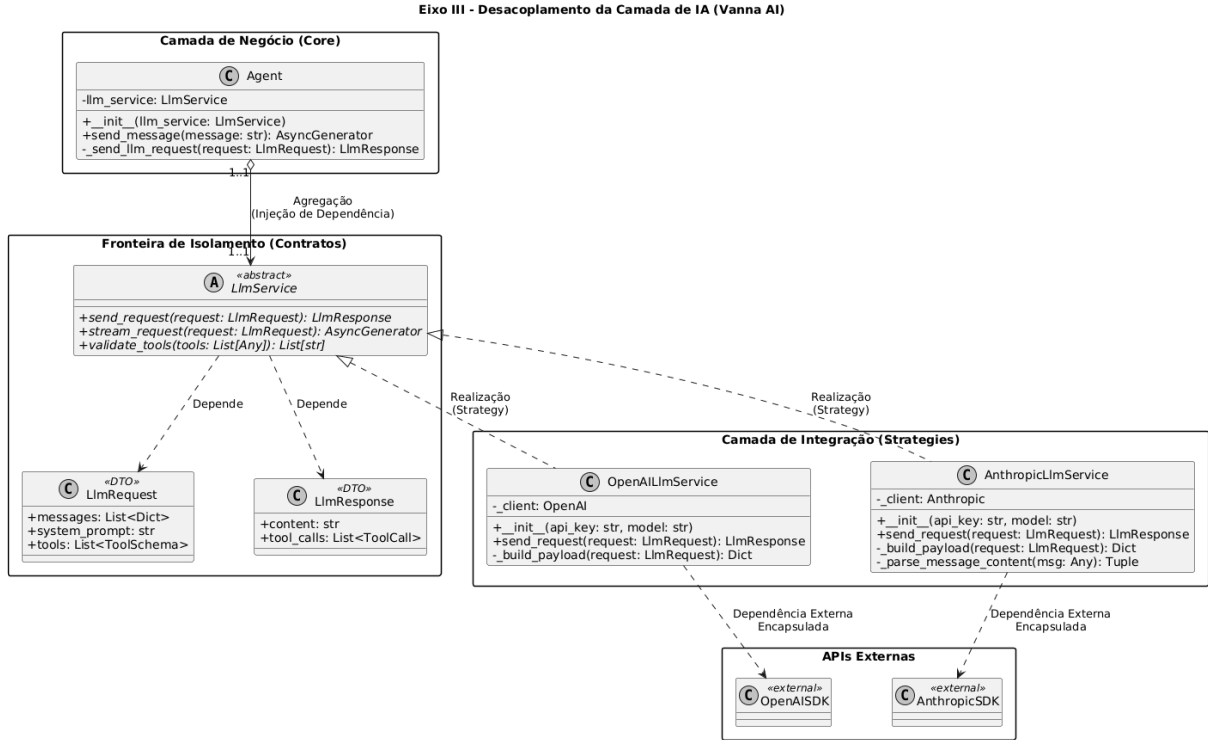


Figura 1: Diagrama de Classes: Desacoplamento da Camada de IA via *Strategy* e Injeção de Dependência.

O diagrama ilustra como as dependências aos SDKs de terceiros são encapsuladas na camada de integração, enquanto o *Core* comunica-se exclusivamente com a abstração *LlmService*. Adicionalmente, foi identificada a implementação do padrão **Adapter** (`src/vanna/legacy/adapter.py`), que envelopa lógicas do monolito legado (*VannaBase*) para adequá-las ao novo barramento de ferramentas, comprovando um tratamento maduro da dívida técnica.

5.3 Alinhamento com Modelos de Maturidade

- **CMMI-DEV v2.0 (Solução Técnica - TS):** O projeto demonstra maturidade na prática de “Selecionar e Projetar Soluções” ao isolar componentes tecnológicos voláteis. A arquitetura permite incorporar novas IAs no futuro sem alterar o núcleo de orquestração, mitigando riscos severos de dependência de fornecedores (*vendor lock-in*).
- **MPS.BR (Projeto e Construção - PJR):** Conforme exigido no Nível G, o software descreve suas interfaces de forma precisa. O uso de tipagem estática e abs-

trações garante que a construção seja rastreável e fortemente desacoplada, atestando a alta manutenibilidade da solução técnica.

6 Eixo IV: Verificação e Validação - V&V (Foco: V&V)

Este eixo analisa se o software foi implementado como planejado (Verificação) e se atende às expectativas do usuário (Validação), requisitos críticos tanto para CMMI (VER/VAL) quanto MPS.BR.

6.1 Análise de Verificação (Workflows e Testes)

Foram identificados fortes mecanismos de Integração Contínua (CI) na pasta `.github/workflows`. O principal workflow estrutural é o `tests.yml`.

Workflow Automático:

on:

push:

branches:

- main

Isso assegura que toda modificação estrutural dispara pipelines automáticas. A execução é gerenciada pela ferramenta `tox`, validando múltiplos ambientes:

```
- name: Install pip
  run: |
    python -m pip install --upgrade pip
    pip install tox
- name: Run tests
  run: tox
```

Foi também notado que as execuções de teste injetam credenciais via Github Secrets (`${{ secrets.OPENAI_API_KEY }}`), sugerindo que os testes efetuam requisições ponta a ponta (E2E) com as APIs reais. Outro workflow relevante, `python-publish.yml`, atua na publicação agnóstica de versões, auxiliando na verificação de empacotamento.

Estrutura de Testes: A suíte de verificação é modular e densa:

- `conftest.py`: Fixtures e ambiente de execução.
- `test_agent_memory.py`: Valida retenção de estado da IA.
- `test_workflow.py`: Valida fluxos de execução determinísticos.
- Testes de adaptadores: `test_azureopenai_llm.py`, `test_gemini_integration.py`, `test_ollama_direct.py`.

Embora automação técnica exista, a **rastreabilidade entre os requisitos de teste e o código é inexistente**, caracterizando aderência apenas *parcial* ao MPS.BR/CMMI.

6.2 Análise de Validação e Prevenção de Alucinações

Em nível de código, o processo de validação é colaborativo via GitHub (Peer Review): o repositório possuía (no momento da auditoria) 64 PRs abertos e 273 concluídos.

Avaliando a Prevenção de Alucinações (Validação da saída da IA): **Foi constatada uma grave lacuna metodológica**. Apesar de arquitetura modular, o sistema **não possui pipelines defensivas** para verificação rigorosa da sintaxe SQL antes da execução cega nos bancos de dados (além da captura simples de erro do SGDB). Encontrou-se uma pasta `evals` (`llm_comparison.py`, `benchmarks`), porém esta trata-se de experimentação offline (Benchmarking), não de validação contextual em tempo de execução (Produção). O sistema falha em adequar-se integralmente ao nível V&V.

7 Eixo V: Qualidade de Software (Foco: GQA)

O eixo de Qualidade (CMMI: PPQA / MPS.BR: Garantia da Qualidade) examina a adoção rigorosa de métricas estáticas e padrões de desenvolvimento.

7.1 Auditoria do CONTRIBUTING.md

O guia de contribuições do repositório, apesar de estar arquivado desde março de 2026, estipula diretrizes formidáveis. Requisita Python 3.11+, uso de ambientes virtuais obrigatórios e possui um pipeline estrito:

- Uso do **Ruff** para linting e formatação (`ruff format ...`, `ruff check ...`).
- Uso do **Mypy** obrigatório para tipagem estática (`tox -e mypy`).
- Padronização forte (*Conventional Commits*): `feat:`, `fix:`, `refactor:`, etc.

7.2 Dívida Técnica Analisada

A transição para a "Versão 2.0" gerou um gargalo estrutural, exigindo manter dois barramentos paralelos (*VannaBase* legado e framework *Agent-based*). A tabela abaixo evidencia o peso atual do repositório:

Indicador	Valor	Impacto na Dívida
Issues abertas	227	Alto (Sinal de acúmulo de requisições)
Bugs não corrigidos	99	Alto
PRs aguardando revisão	64	Médio (Gargalo de Code Review)
Releases publicadas	78	Positivo (Ritmo ativo)
Contribuidores	72	Positivo (Base comunitária ampla)

Tabela 1: Visão Geral da Dívida Técnica

A análise estática do `pyproject.toml` detectou dívida intencional documentada. Mais de 25 regras vitais do Ruff estão com "ignore", com justificativas como "legacy compatibility" ou "intentional in legacy code" (ex: regras F841, B027, B904). Ademais, as dependências nucleares (pandas, PyYAML, sqlalchemy) estão listadas sem *version pinning* seguro, acarretando sério risco de regressões.

Outro ponto crítico é a **ausência de ferramentas SAST** (Static Application Security Testing), como *CodeQL* ou *Bandit*. Em um projeto que executa código gerado por IA (SQL), não mitigar o risco de *SQL Injection* no pipeline de CI é um gap massivo de qualidade.

7.3 Avaliação segundo Modelos de Maturidade

Critério Auditado	Status	Nível / CMMI MPS.BR (Projeção)
CONTRIBUTING.md Estruturado	Conforme	Nível 2 - Gerenciado
Linting e formatação automatizados	Conforme	Nível 2 - Gerenciado
Type checking estrito (Mypy)	Conforme	Nível 3 - Definido
Pre-commit hooks implementados	Conforme	Nível 2 - Gerenciado
Ferramenta de cobertura de testes	Parcial (Sem Meta)	Abaixo do Nível 2 (Gap de Controle)
Análise SAST (Segurança) automatizada	Ausente	Abaixo do Nível 2
Gestão formal de dívida técnica	Ausente	Abaixo do Nível 2

Tabela 2: Aderência do Projeto Vanna aos Modelos de Maturidade

8 Plano de Melhoria de Processo

Como corolário da auditoria técnica, formulamos um plano estratégico visando retificar as lacunas organizacionais para que o Vanna AI alcançasse os parâmetros elementares de

certificação do nível inicial (Nível G) do MPS.BR.

8.1 Síntese dos Gaps e Critérios de Priorização

A auditoria revelou que as engenharias de base técnica (qualidade de código em Python) são robustas, contudo, os fundamentos de processo gerencial (GPR e GRE) são embrionários. O projeto carece de planejamento tático, gestão sistemática de rastreabilidade, e mitigação preventiva de riscos de arquitetura.

As ações foram priorizadas obedecendo o alinhamento com MPS.BR Nível G, visando impacto imediato na saúde do projeto com baixíssimo atrito de implementação na cultura open source.

8.2 Ação 1: Gerência de Requisitos com Rastreabilidade Formal (Foco: GRE)

Problema Central: Ausência de formalização para aprovação de novas bibliotecas sistêmicas (RFCs) e documentação falha no mapeamento entre Issue → Testes → Código.

Medidas Propostas:

- Criação de templates de issues restritivos (`BUG_REPORT.md`, `FEATURE_REQUEST.md`) que obriguem o reportador a fornecer Critérios de Aceitação.
- Implementação de um repositório formalizado de RFCs em markdown.
- Estabelecimento de um Tracker (ex. `TRACEABILITY.md`) para auditar PRs críticos.

8.3 Ação 2: Gerência de Projetos com Controle de Riscos (Foco: GPR)

Problema Central: O desenvolvimento ocorre reativamente. Os riscos de dependência de APIs voláteis (Ex: Mudanças na precificação ou deprecamento de end-points da OpenAI) não são acompanhados e tampouco há previsibilidade de roadmaps.

Medidas Propostas:

- Adoção contínua da feature "Milestones" do GitHub para encapsular entregas temáticas.
- Criação e manutenção do `RISK_REGISTER.md`, onde vulnerabilidades de arquitetura (como ausência de SAST) e riscos de negócio (APIs) possam ser ativamente mitigados e discutidos.

8.4 Plano de Execução e Métricas

A introdução cultural dessas rotinas em um time open source deve ser cadenciada. Sugere-se uma implantação de 4 semanas.

- **Semana 1:** Disponibilização de templates e RFCs.
- **Semana 2 e 3:** Introdução da política de Rastreabilidade nos novos Pull Requests e catalogação dos riscos (`RISK_REGISTER.md`).
- **Semana 4:** Transição para Milestones fechados.

A avaliação de eficácia será auferida rastreando a porcentagem de PRs ligados a issues devidamente categorizadas, e através de métricas de taxa de cumprimento temporal dos Milestones.

8.5 Conclusão do Plano

Se aplicadas diligentemente, as ações propostas mitigarão de forma estrutural as lacunas detectadas. O estado futuro do Vanna representaria um repositório maduro não somente tecnicamente, mas formalizado nos pilares de auditoria, tornando o ecossistema plenamente elegível para atuar em missões críticas corporativas com resguardo CMMI Nível 2 / MPS.BR Nível G.